surrounding.c

Enumeration of Hat-Tile Surroundings of the Anti-Hat

`MyHatTileStuff/surrounding.c` — Technical Reference

June 14, 2026

Purpose

`surrounding` enumerates all distinct ways to place unreflected hat tiles around a fixed *anti-hat* center tile so that:

- every surrounding tile shares at least one edge directly with the center (direct contact, not just corner or proximity),
- the union of all tiles — center plus surrounding — has no holes (no enclosed empty regions), and
- at least $n_{\min}$ surrounding tiles are present (default: 5; controllable via `-min`).

Two configurations are considered the *same* if one can be obtained from the other by a symmetry of the tetrille lattice (any combination of rotations and reflections). The result is a set of symmetry classes visualised as an SVG grid.

The motivation is to characterise which local arrangements of regular (unreflected) hat tiles can appear around an anti-hat in a valid hat tiling, supporting the broader research goal of PolyformTown.

Background: Hat Tile and Anti-Hat

The Hat Tile

The hat is a 13-edge polygon on the **tetrille** lattice, first described in 2023 as the first known aperiodic monotile. The tetrille lattice is a kite-and-dart decomposition of the plane with three vertex valences: $v \in \{3, 4, 6\}$. Coordinates are integer triples (v, x, y) where v carries the valence tag; cross-tag arithmetic is impossible without explicit lifting formulas.

The hat tile has 14 vertices in its boundary cycle (the 13-edge polygon closes back to the start), using all three valence systems. Its `hat.tile` DSL entry begins:

```
name: hat
lattice: tetrille
constants:
r = sqrt(3)
cycle:
6  0  0   4 -1  0   3 -1  0
4 -1 -1   6  0 -1   4  0 -3
3 -1 -2   4  1 -3   3  0 -2
4  1 -2   6  1 -1   4  2 -1
3  1 -1   4  1 -1
```

Chirality: Hat vs. Anti-Hat

The hat tile is *chiral*: it and its mirror image (the **anti-hat**) are geometrically distinct and cannot be superimposed by any rotation. Together they tile the plane aperiodically in a fixed ratio of approximately 3:1 (hats to anti-hats).

The tetrille lattice's symmetry group has 12 elements: transforms $t = 0 \ldots 5$ are pure rotations; $t = 6 \ldots 11$ are a fixed reflection followed by a rotation. Applying $t = 6$ (pure reflection) to the base hat cycle produces the anti-hat.

`tile_build_variants()` computes all 12 symmetry images of the base cycle and deduplicates them. For the hat tile this yields exactly 12 distinct variants (6 hat orientations and 6 anti-hat orientations) because neither chirality has any non-trivial rotational symmetry.

Data Structures

Core Types

TileCyc and Config

TileCyc stores one tile's boundary cycle compactly enough for SVG rendering without retaining a full `Cycle` structure. It records the vertex array, its length, and a chirality flag (retained for the centre tile even though surrounding tiles are always unreflected hats).
Config records one complete configuration: the count of surrounding tiles and their `TileCyc` entries. The centre is always `g_center` (implicit), so it is not stored in the config.

```c
#define HAT_NMAX  20   /* 14 verts + margin */
#define MAX_SUR   18   /* DFS depth cap     */
#define MAX_CFGS  5000 /* output cap        */

typedef struct {
    int   n;
    Coord v[HAT_NMAX];
    int   reflected; /* 1=anti-hat, 0=hat */
} TileCyc;

typedef struct {
    int     sur_count;
    TileCyc sur[MAX_SUR];
} Config;
```

Global State

Global Variables

The program keeps all mutable state in file-scope statics so the DFS function signature stays clean. The most critical globals are the per-depth aggregate array `g_agg` and the DFS tile stack `g_path`.

`g_tile`
The loaded tile (hat), including all 12 precomputed variants.
`g_refl[vi]`
Chirality flag: 1 if variant `vi` is an anti-hat.
`g_center`
The anti-hat centre cycle (fixed, never changes).
`g_cedges[], g_cedge_n`
The 14 directed boundary edges of the centre tile; used to identify which aggregate boundary edges are eligible for attaching a surrounding tile.
`g_seen`
A `HashTable` of canonical aggregate `Poly` shapes; each canonical state is visited at most once.
`g_cfgs[], g_cfg_n`
The collected output configurations.

```c
static Tile      g_tile;
static int       g_refl[MAX_VARIANTS];
static Cycle     g_center;
static Edge      g_cedges[32];
static int       g_cedge_n;
static HashTable g_seen;
static Config    g_cfgs[MAX_CFGS];
static int       g_cfg_n;
static int       g_min_sur;
static int       g_maximal_only;

/* DFS tile stack: g_path[d] = tile placed at depth d
↪  */
static TileCyc   g_path[MAX_SUR];

/* Per-depth aggregate polys (see Section 5) */
static Poly      g_agg[MAX_SUR + 2];

/* Scratch temporaries (reused, never aliased) */
static Poly      g_canon_tmp;
static Cycle     g_aligned_tmp;
```

Initialisation

Loading and Variant Classification

Chirality Identification — `identify_reflected()`

After `tile_load()` populates `g_tile`, the program must determine which of the 12 variants are unreflected hats and which are anti-hats.
The strategy: for each variant, apply all six pure-rotation transforms ($t = 0 \ldots 5$) to the *base* cycle and check if any result is coordinate-equal to that variant. A variant reachable by a pure rotation is a hat; all others are anti-hats. Cycles are normalised (position and start vertex) before comparison so that coordinate equality is well-defined regardless of the lattice origin or cycle winding offset.

```
static void identify_reflected(void) {
  for (int vi = 0; vi < g_tile.variant_count; vi++) {
    g_refl[vi] = 1; /* assume anti-hat */
    for (int t = 0; t < 6; t++) {
      Cycle tmp;
      cycle_transform_lattice(
          &g_tile.base, &tmp,
          g_tile.lattice, t);
      if (cycle_signed_area2(&tmp, g_tile.lattice) <
      ↪  0)
          cycle_reverse(&tmp);
      cycle_normalize_position(&tmp, g_tile.lattice);
      /* coordinate-equal → it is a pure rotation */
      if (!cycle_less(&tmp, &g_tile.variants[vi]) &&
          !cycle_less(&g_tile.variants[vi], &tmp)) {
          g_refl[vi] = 0; /* it's a hat */
          break;
      }
    }
  }
}
/* Result for hat.tile: 6 hats (g_refl=0),
                        6 anti-hats (g_refl=1) */
```

Centre Construction

Anti-Hat Centre and Edge Extraction

The centre tile is constructed by applying transform $t = 6$ (the tetrille reflection) to the base hat cycle, normalising orientation (ensure CCW) and position. This produces the canonical anti-hat in a fixed, reproducible coordinate frame.
All 14 directed boundary edges of the centre are stored in `g_cedges[]` at startup. During DFS only aggregate boundary edges that appear in this set are eligible attachment sites, enforcing the direct-contact requirement.

```
/* Build the anti-hat centre */
{
  Cycle tmp;
  cycle_transform_lattice(
      &g_tile.base, &tmp,
      g_tile.lattice, 6);      /* t=6 = pure
      ↪  reflection */
  if (cycle_signed_area2(&tmp, g_tile.lattice) < 0)
      cycle_reverse(&tmp);      /* ensure CCW */
  cycle_normalize_position(&tmp, g_tile.lattice);
  g_center = tmp;
}

/* Seed aggregate: centre alone */
g_agg[0].cycle_count = 1;
g_agg[0].cycles[0]   = g_center;

/* Record the 14 centre boundary edges */
{
  Edge tmp[64];
  g_cedge_n = build_boundary_edges(&g_agg[0], tmp);
  memcpy(g_cedges, tmp,
      (size_t)g_cedge_n * sizeof(Edge));
}
```

Depth-First Search

Overview

The enumeration proceeds by depth-first search over sequences of tile placements. The DFS state at depth d is the aggregate polygon `g_agg[d]`, which is the union of the centre tile plus the d surrounding tiles placed so far. At each depth the DFS:

1. canonicalises `g_agg[d]` and inserts it into `g_seen`; if already seen, returns immediately (dedup),
2. enumerates all valid one-tile extensions subject to the two constraints (see Section 6),
3. recurses on each extension at depth $d+1$,
4. after all extensions are tried, records the current state as a finished configuration if it qualifies.

Per-Depth Aggregate Stack

Stack Layout — `g_agg[0..MAX_SUR+1]`

A `Poly` struct is large (≈ 144 KB for `MAX_CYCLES=32`, `MAX_VERTS=384`). Placing one on the C call stack at each recursive level would risk a stack overflow at depth 18.

The solution is a *pre-allocated global array* indexed by depth. `g_agg[d]` is the aggregate at depth d. When the DFS at depth d calls `try_attach_tile_poly_ex`, it passes `&g_agg[d+1]` as the output buffer. The recursive call at depth $d{+}1$ then reads `g_agg[d+1]` as its starting state and writes to `g_agg[d+2]`, and so on.

Because a level d only modifies `g_agg[d+1]`, `g_agg[d]` remains valid across the recursive call. Backtracking is free: the next extension at depth d simply overwrites `g_agg[d+1]` with a different placement.

```
g_agg[0]  = centre tile only       (depth 0, set in
↪  main)
g_agg[1]  = centre + tile[0]       (written at depth
↪  0)
g_agg[2]  = centre + tile[0,1]     (written at depth
↪  1)
   ...
g_agg[d]  = centre + tile[0..d-1]
g_agg[d+1]= candidate new state    (written, then read
                                    at depth d+1)
```

```
/* DFS writes to g_agg[depth+1] before recursing */
if (!try_attach_tile_poly_ex(
      agg,                    /* g_agg[depth]   */
      &g_tile.variants[vi],
      g_tile.lattice,
      be, te,
      &g_agg[depth + 1],     /* output: next level */
      &g_aligned_tmp))
    continue;
dfs(depth + 1);
/* On return, g_agg[depth] is unchanged;
   g_agg[depth+1] will be overwritten by next
   iteration                                       */
```

Deduplication

State Deduplication via Canonical Hash

Many different tile-insertion *sequences* produce the same aggregate *shape*. Without deduplication the DFS would explore exponentially many paths to the same state.

At the top of `dfs(depth)`, *before* any extension is tried, the aggregate `g_agg[depth]` is canonicalised under all 12 tetrille symmetries and the result is inserted into `g_seen`. If `hash_insert` returns 0 (already present), the call returns immediately.

This is the standard "canonical form at the root" trick: since the canonical form is the same regardless of insertion order, each equivalence class of aggregate shapes is explored exactly once.

```c
static void dfs(int depth) {
  /* Canonicalise and dedup */
  poly_hash_key_lattice(
      &g_agg[depth],
      g_tile.lattice,
      &g_canon_tmp);          /* 12-transform minimum
      ↪  */
  if (!hash_insert(&g_seen, &g_canon_tmp))
      return;                 /* already explored */

  if (depth >= MAX_SUR) {
      if (depth >= g_min_sur) store_config(depth);
      return;
  }
  /* ... extension loop ... */
}

/* poly_hash_key_lattice applies all 12 tetrille
   transforms, normalises each image, and returns
   the lexicographically smallest one.  Two polys
   get the same canonical form iff they belong to
   the same symmetry class.                        */
```

Extension Loop

Tile Attachment — Extension Loop

For each boundary edge `be` of the current aggregate:

1. Skip `be` if it is not a centre edge (`is_center_edge()` — ensures direct contact).
2. For each unreflected hat variant `vi` (anti-hats skipped via `g_refl[vi]`):
3. For each edge index `te` of variant `vi`:
4. Call `try_attach_tile_poly_ex` to attempt placing variant `vi` with its edge `te` antiparallel to aggregate edge `be`.
5. If the result has holes (`cycle_count > 1`), skip.
6. Otherwise record the placed tile in `g_path[depth]` and recurse.

The flag `extended` tracks whether any valid extension was found, used later for the maximal-mode filter.

```
const Poly *agg = &g_agg[depth];
Edge edges[256];
int ec = build_boundary_edges(agg, edges);

int extended = 0;
for (int be = 0; be < ec; be++) {
  if (!is_center_edge(edges[be])) continue;

  for (int vi = 0; vi < g_tile.variant_count; vi++) {
    if (g_refl[vi]) continue;  /* hats only */

    for (int te = 0; te < g_tile.variants[vi].n; te++)
    ↪ {
      if (!try_attach_tile_poly_ex(
              agg, &g_tile.variants[vi],
              g_tile.lattice, be, te,
              &g_agg[depth + 1], &g_aligned_tmp))
          continue;

      /* Constraint: no holes */
      if (g_agg[depth + 1].cycle_count > 1)
          continue;

      extended = 1;
      /* Copy placed tile into path stack */
      int n = g_aligned_tmp.n;
      if (n > HAT_NMAX) n = HAT_NMAX;
      g_path[depth].n         = n;
      g_path[depth].reflected = g_refl[vi]; /* =0 */
      for (int i = 0; i < n; i++)
          g_path[depth].v[i] = g_aligned_tmp.v[i];

      dfs(depth + 1);
    }
  }
}
```

Configuration Recording

After the extension loop, the current depth-d state is recorded if it satisfies the output criteria:

$$\texttt{depth} \geq g\_min\_sur \quad \text{and} \quad (\neg\texttt{g\_maximal\_only} \vee \neg\texttt{extended}).$$

In default mode every state with at least g_min_sur tiles is stored. In `-maximal` mode only states from which no further valid hat tile can be placed (touching the centre, without creating a hole) are stored.

Key Constraints

Direct Contact with the Centre

Every surrounding tile must share a boundary edge with the centre anti-hat, not merely be adjacent to a previously placed surrounding tile.

This is enforced by `is_center_edge(e)`: it returns 1 only if the directed edge e appears in the pre-recorded centre edge set `g_cedges[]`. Because tiles are attached to *boundary* edges of the growing aggregate, and because a centre edge disappears from the aggregate boundary once a tile is attached there, this filter ensures that every tile in the DFS is placed directly against the centre.

```
static int is_center_edge(Edge e) {
    for (int i = 0; i < g_cedge_n; i++)
        if (coord_eq(e.a, g_cedges[i].a) &&
            coord_eq(e.b, g_cedges[i].b))
            return 1;
    return 0;
}
```

Note: `coord_eq` checks all three fields (v, x, y), so the comparison is exact and tag-safe.

Unreflected Hats Only

Anti-hat tiles are excluded from the surrounding by the single guard:

```
if (g_refl[vi]) continue;
```

inside the variant loop. Only the six unreflected hat variants ($t = 0 \ldots 5$) are ever tried as surrounding tiles.

No-Hole Filter

The `Poly` struct produced by `try_attach_tile_poly_ex` can have multiple boundary cycles: `cycles[0]` is always the outer boundary (CCW), and `cycles[1...]` are hole boundaries (CW). A `cycle_count` greater than 1 signals that the placement would enclose an empty region.

```
if (g_agg[depth + 1].cycle_count > 1) continue;
```

This check is applied *immediately after* attachment, before recursing. Holes are monotone: once created, no additional tiles on the exterior can eliminate them. Early pruning therefore cuts entire subtrees, not just individual leaf configurations.

Tile Attachment Mechanics

`try_attach_tile_poly_ex` is the geometric core from `src/core/attach.c`. Given aggregate `agg`, a tile variant cycle `tv`, boundary edge index `be`, and tile edge index `te`, it:

1. **Aligns** the tile so that the directed tile edge `tv.v[te]` $\rightarrow$ `tv.v[te+1]` is antiparallel and coincident with the directed aggregate edge `edges[be]`. The required translation is computed in the 6-system (the coarsest integer lattice that contains all three valence systems) to avoid cross-tag arithmetic.
2. **Collects** all directed edges from both the aggregate and the aligned tile into a single flat list.
3. **Cancels** pairs of antiparallel coincident edges (the shared edges that disappear when two tiles are joined). Parallel coincident edges indicate overlap and abort with `ATTACH_STATUS_GEOMETRY`.
4. **Walks** the remaining directed edges to extract closed boundary cycles, classifying each by signed area: $A_2 > 0$ (CCW) $\Rightarrow$ outer; $A_2 < 0$ (CW) $\Rightarrow$ hole.
5. Writes the aggregate result to `g_agg[depth+1]` and the aligned tile's own cycle to `g_aligned_tmp`.

The `g_aligned_tmp` output is the tile in the shared coordinate frame — its vertex array is copied into `g_path[depth]` for later SVG rendering.

Canonicalisation

`poly_hash_key_lattice(src, lattice, key)` reduces a `Poly` to its canonical representative under the 12-element tetrille symmetry group (D_6 acting on the tagged integer lattice):

1. For each transform $t \in \{0, \ldots, 11\}$:
   (a) Apply `poly_transform_lattice(src, &tmp, lattice, t)`.
   (b) Translate `tmp` so its lexicographically smallest vertex is at the system-appropriate origin (`poly_normalize_position`).
   (c) Rotate each cycle to start at its own lexicographically smallest vertex (`cycle_canonicalize_shift`).
   (d) Sort hole cycles lexicographically.

2. Select the transform image `tmp` that is lexicographically smallest overall as the canonical key.

Two `Poly` shapes hash to the same key if and only if they are related by a tetrille symmetry, which is exactly the condition for them being *equivalent* surroundings. The hash table `g_seen` uses this key; `hash_insert` returns 0 on collision, preventing re-exploration of equivalent aggregate states.

Consequence for DFS. The dedup check fires at the *start* of each DFS call, before any new tile is placed. This means the state *before extension* is tested — equivalently, the set of all partial placements up to and including depth d is represented by a single canonical aggregate rather than by the (exponential) set of tile-insertion sequences that produce it.

SVG Rendering

Coordinate Conversion

Tetrille lattice coordinates (v, x, y) are mapped to real screen coordinates using the basis matrices from `hat.tile`. The $\sqrt{3}$ factor is pre-computed once as `g_r3 = sqrt(3.0)`.

$$\begin{pmatrix} x_{\text{screen}} \\ y_{\text{screen}} \end{pmatrix} = \begin{cases} \begin{pmatrix} \frac{\sqrt{3}}{2}(x+y) \\ \frac{1}{2}(y-x) \end{pmatrix} & v = 6 \\ \begin{pmatrix} \frac{\sqrt{3}}{4}(x+y) \\ \frac{1}{4}(y-x) \end{pmatrix} & v = 4 \\ \begin{pmatrix} \frac{1}{\sqrt{3}}x + \frac{1}{2\sqrt{3}}y \\ \frac{1}{2}y \end{pmatrix} & v = 3 \end{cases}$$

SVG y increases downward while mathematical y increases upward, so the screen y-coordinate is negated: a vertex at logical (x_s, y_s) maps to SVG position $(o_x + \texttt{SVG\_UNIT} \cdot x_s,\ o_y - \texttt{SVG\_UNIT} \cdot y_s)$ where (o_x, o_y) is the cell origin offset.

Layout

Configurations are sorted by surrounding tile count and arranged in a grid whose aspect ratio approximates 16:9:

- A single global bounding box is computed over all tile vertices in all configurations, ensuring all cells have the same scale.
- Each cell receives a label "`n=k`" below the drawing.
- Surrounding tiles are drawn first (behind the centre); the centre anti-hat is drawn on top.

Colour coding:

- **Coral** (`#e74c3c`): centre anti-hat.
- **Light blue** (`#a8d8ea`): surrounding (unreflected) hat tiles.

Results

Running `./bin/surrounding` with the default hat tile produces:

Mode	Configurations	Distribution
Default (`-min 4`)	108	all $n = 4$
`-maximal`	108	all $n = 4$

The identical counts in both modes reveal that every valid 4-hat surrounding is already maximal: no 5th unreflected hat tile can be attached to any remaining centre edge without either overlapping an existing tile or creating a hole. This is a non-trivial geometric constraint arising from the specific shape of the anti-hat and the chirality restriction.

Diagnostic output printed to `stderr`:

```
=== Hat Tile Surroundings ===
Tile file:       tiles/hat.tile
Variants:        12 total (6 hat, 6 anti-hat)
Center edges:    14
Min surrounding: 4
Mode:            maximal only

Configurations:  108
States visited:  337
```

Usage

```
# Build
make surrounding

# All configurations with >= 4 surrounding hats
./bin/surrounding

# Only maximal configurations
./bin/surrounding --maximal

# Custom minimum, custom output file
./bin/surrounding --min 3 --out my.svg

# Different tile file
./bin/surrounding --tile tiles/hat.tile --out surrounding.svg
```

The output SVG is written to `surrounding.svg` (or the path given by `-out`). It is self-contained and can be opened in any standards-compliant SVG viewer or web browser.

Architecture Summary

```
tile_load("tiles/hat.tile")
  +-- tile_build_variants() -> 6 hat + 6 anti-hat variants
         |
         +-- identify_reflected() -> g_refl[vi]
         |
         +-- cycle_transform_lattice(base, t=6) -> g_center (anti-hat)
         |
         +-- build_boundary_edges(g_agg[0]) -> g_cedges[14]

dfs(depth)
  +-- poly_hash_key_lattice(g_agg[depth]) -> g_canon_tmp
  |    +-- if seen: return (dedup)
  |
  +-- build_boundary_edges(g_agg[depth]) -> edges[]
  |
  +-- for each edge be:
       +-- if is_center_edge(be):       [direct contact]
            +-- for each variant vi (g_refl[vi]==0): [hats only]
                 +-- for each tile edge te:
                      +-- try_attach_tile_poly_ex(...)
                           +-- if fail: skip
                           +-- if cycle_count > 1: skip [no holes]
                           +-- copy g_aligned_tmp -> g_path[depth]
                                dfs(depth + 1)

  +-- if depth >= g_min_sur && (all||!extended): store_config(depth)

emit_svg() -> surrounding.svg
```

`MyHatTileStuff/surrounding.c` — part of PolyformTown. Compile this document with `pdflatex -shell-escape surrounding.tex` (requires `pygments` for `minted`).